

コマ 4: 変数・代入・シンボルテーブル

今日のゴール

ローカル変数の宣言・参照・代入を実装する。

第 03 回までは、`return 1 + 2 * 3;` のように、式の中に数値しか登場しなかった。この回からは、次のようなプログラムを扱う。

```
int main() {  
    int a;  
    int b;  
    int c;  
    a = 3;  
    b = 5;  
    c = a + b;  
    return c;  
}
```

目標は、変数 `a`, `b`, `c` をスタック上に確保し、代入・参照できるようにすることである。

1 この回で扱う範囲

対象にするプログラムは、次の形に限定する。

```
int main() {  
    int 変数名;  
    ...  
    文;  
    ...  
}
```

扱う機能は以下の通り。

種類	例
ローカル変数宣言	<code>int a;</code>
変数への代入	<code>a = 3;</code>

種類	例
変数参照	<code>return a;</code>
変数を含む算術式	<code>c = a + b * 2;</code>
複数文	<code>a = 3; b = 5; return a + b;</code>

この回では、すべての変数を 8 バイト整数として扱う。`int` は本来 4 バイトだが、前半では実装を単純にするため、スタック上では各変数に 8 バイトを割り当てる。

`if`、`while`、関数呼び出し、ポインタ、配列はまだ扱わない。

2 AST を確認する

まず、変数を含むプログラムがどのような AST になるか確認する。

```
python3 scaffold/parse_viewer.py sessions/04_variables/tests/target.c
```

このプログラムの内容は次の通り。

```
int main() {
    int a;
    int b;
    int c;
    a = 3;
    b = 5;
    c = a + b;
    return c;
}
```

実行すると、次のような S 式が表示される。

```
(program
  (funcdef "main" :type int (params)
    (block (decl "a" :type int) (decl "b" :type int) (decl "c" :type int)
      (exprstmt
        (assign (var "a") (num 3)))
      (exprstmt
```

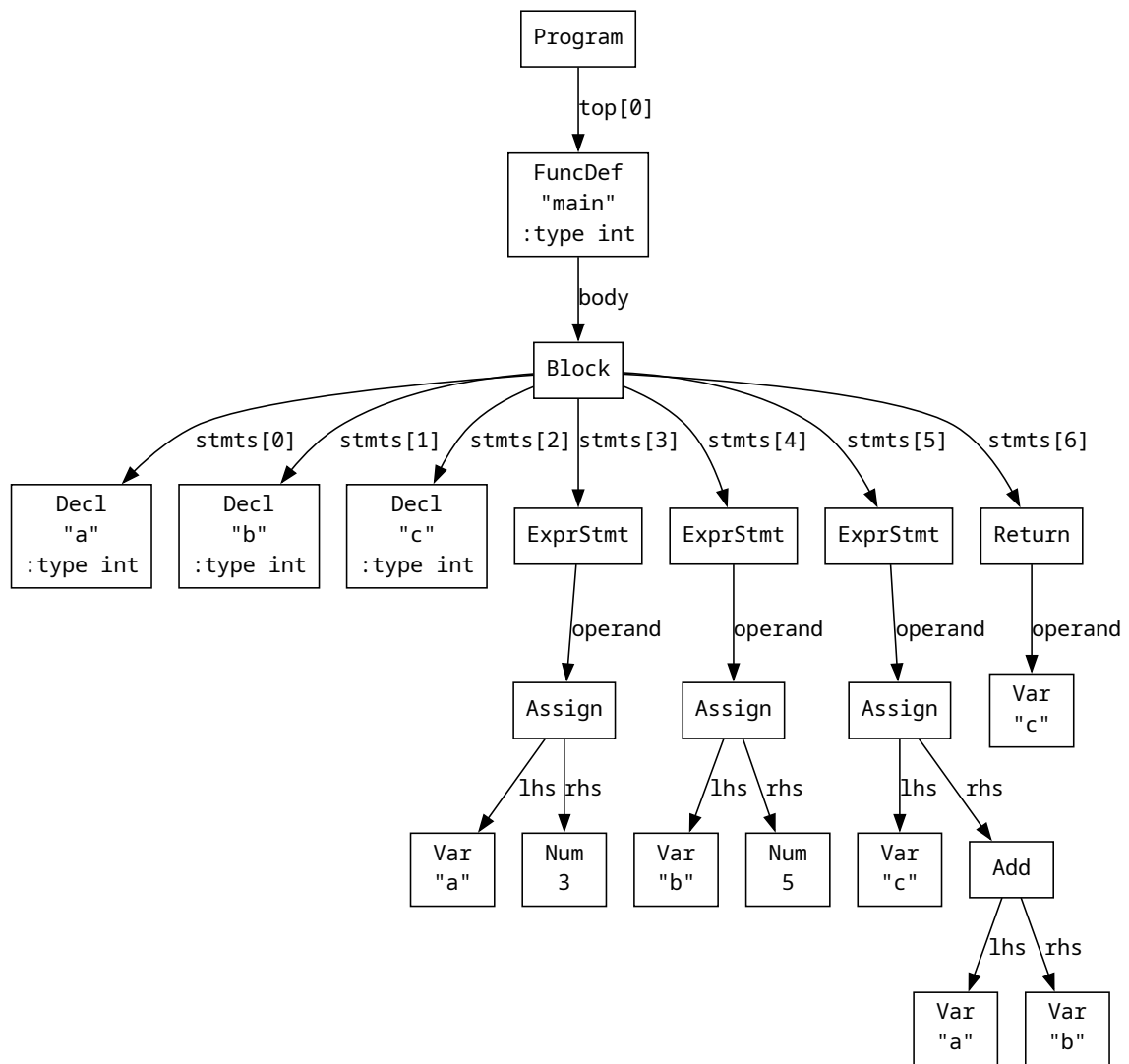


図 1 変数を含むプログラムの AST

```

(assign (var "b") (num 5)))
(exprstmt
  (assign (var "c")
    (add (var "a") (var "b"))))
(return (var "c"))))

```

新しく登場するノードは次の通り。

S 式	Node での表現	意味
(decl "a" :type int)	ND_DECL	ローカル変数 a の宣言

S 式	Node での表現	意味
(var "a")	ND_VAR	変数 a の参照
(assign A B)	ND_ASSIGN	A = B
(exprstmt X)	ND_EXPRSTMT	X; という式文
(block ...)	ND_BLOCK	{ ... } の中の文列

return c; の c も (var "c") として表される。a = 3; の左側の a も、右辺で値を読む a も、AST 上は同じ ND_VAR である。

3 式と文

ここからは、「式」と「文」を区別する必要がある。

式とは、評価すると結果の値が得られるものである。得られた値は、別の計算に使い回せる。

例:

```
1 + 2
a
a + b * 3
a = 10
```

たとえば、1 + 2 は評価すると 3 になる。その結果は、さらに別の式の一部として使える。

```
(1 + 2) * 4
```

この式では、まず 1 + 2 の結果 3 が得られ、その値を使って 3 * 4 を計算する。

変数参照 a も式である。評価すると、変数 a に保存されている値が得られる。

代入 a = 10 も式である。代入は a に 10 を書き込む処理を行い、さらに式全体の結果として 10 という値を持つ。そのため、C では代入式の結果をさらに使うことができる。

```
b = (a = 10);
```

この場合、a = 10 の結果は 10 なので、b にも 10 が代入される。

一方、文とは、プログラムの中で実行される処理の単位である。文そのものは、次の計算に使い回せる結果の値を返さない。

例:

```
int a;
a = 10;
return a;
```

`a = 10` は式だが、`;` を付けた `a = 10;` は文である。このように、式に `;` を付けて文にしたものを「式文」という。

式文は、式を評価して副作用を起こすために使う。ここでいう副作用とは、変数への書き込みなど、値を返す以外にプログラムの状態を変える処理のことである。

```
a = 10;
```

この文では、代入式 `a = 10` の結果の値 `10` は、それ以上使われない。重要なのは、変数 `a` に `10` が書き込まれるという副作用である。

この回で出てくる文は次の通り。

C コード	AST	意味
<code>int a;</code>	<code>ND_DECL</code>	変数宣言文
<code>a = 10;</code>	<code>ND_EXPRSTMT</code>	式文
<code>return a;</code>	<code>ND_RETURN</code>	return 文
<code>{ ... }</code>	<code>ND_BLOCK</code>	複数の文をまとめるブロック

一方、次のようなものは式である。

C コード	AST	意味
<code>42</code>	<code>ND_NUM</code>	数値
<code>a</code>	<code>ND_VAR</code>	変数参照
<code>a = 10</code>	<code>ND_ASSIGN</code>	代入式
<code>a + b</code>	<code>ND_ADD</code>	足し算

4 codegen と gen_stmt

式と文を区別すると、コード生成のメソッドも分けやすくなる。

`self.codegen(node)` は、式を処理するメソッドである。式には結果の値があるため、`self.codegen(node)` が終わった時点で、その値が `a0` に入っている、という約束にする。

`self.gen_stmt(node)` は、文を処理するメソッドである。文は「処理の単位」であり、次に使い回す値を返す必要はない。そのため、`self.gen_stmt(node)` には「結果を `a0` に残す」という約束は置かない。ただし、`return` 文だけは例外で、関数の戻り値を `a0` に入れる必要がある。

メソッド	入力	役割
<code>self.codegen(node)</code>	式ノード	式を評価し、結果を <code>a0</code> に置く
<code>self.gen_stmt(node)</code>	文ノード	文を実行する命令列を出力する
<code>self.codegen_lval(node)</code>	代入先になる式	書き込み先のアドレスを <code>a0</code> に置く

これらのメソッドはすべて `Codegen04` クラスのインスタンスメソッドである。それぞれ `match node.kind` で処理を分けた後、対応する handler method (例: `gen_stmt_Return`) を呼ぶ。スケルトンではディスパッチ部分はあらかじめ書かれている。

たとえば、`a = 10;` は AST 上では次のように表される。

```
(exprstmt
 (assign (var "a") (num 10)))
```

外側の `exprstmt` は文なので `gen_stmt()` が処理する。内側の `assign` は式なので `codegen()` が処理する。

`return a;` は AST 上では次のように表される。

```
(return (var "a"))
```

外側の `return` は文なので `gen_stmt()` が処理する。内側の `(var "a")` は式なので `codegen()` が処理する。

5 変数はどこに置くか

コンパイル後のプログラムでは、Python の辞書のような場所に変数を置けるわけではない。実行時には、変数の値をメモリ上に保存する必要がある。

この回では、ローカル変数をスタック上に置く。

`Codegen04` には、次のインスタンス変数が用意されている。

```
class Codegen04(prev.Codegen03):
    def __init__(self) -> None:
        super().__init__()
        self._locals: dict[str, int] = {}
        self._stack_offset: int = 0
```

`self._locals` は、変数名からスタック上の位置への対応表である（関数ごとにリセットされる）。

例:

```
{
    "a": -24,
    "b": -32,
    "c": -40,
}
```

この例では、変数 `a` は `s0 - 24`、変数 `b` は `s0 - 32`、変数 `c` は `s0 - 40` に置かれる。

6 スタックフレーム

`gen_func()` は、関数ごとに `_reset_func_state()` を呼んで必要なフレームサイズを計算する。

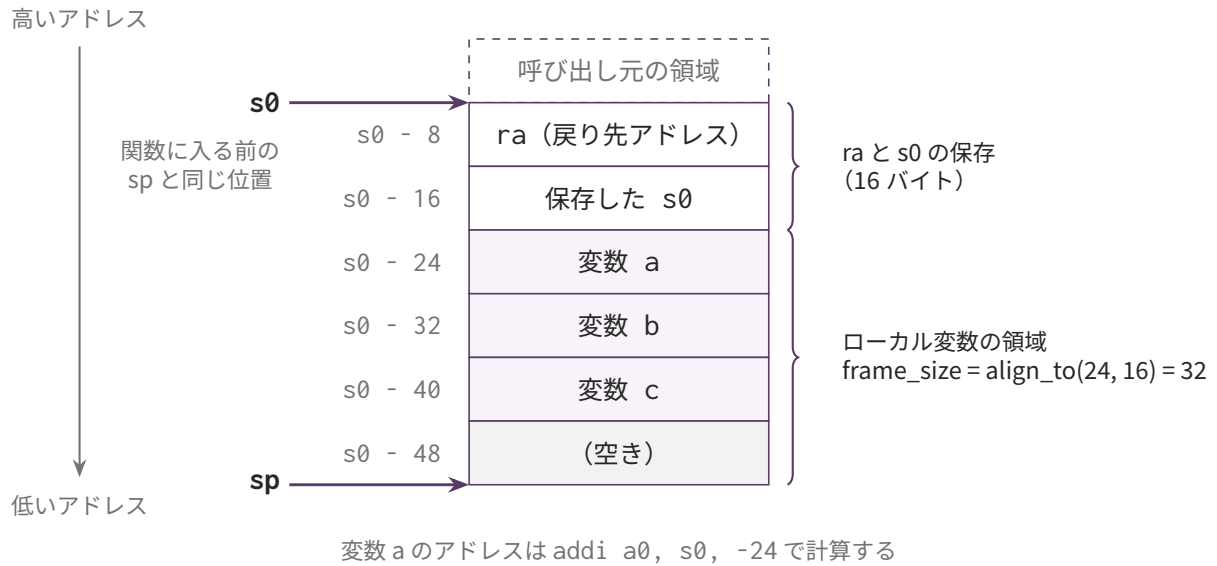


図 2 3 変数 a, b, c を宣言したときのスタックフレーム

```
addi sp, sp, -(frame_size + 16)
sd ra, frame_size + 8(sp)
sd s0, frame_size(sp)
addi s0, sp, frame_size + 16
```

この後、**s0** は関数に入る前の **sp** と同じ位置を指す。ローカル変数は **s0** から負の方向に置く。

ra と保存した **s0** の下にローカル変数が順に並ぶ。1 個目の変数 **a** は **s0 - 24**、2 個目の **b** は **s0 - 32**、3 個目の **c** は **s0 - 40** に置かれる。

コマ 3 では `_reset_func_state()` が 0 を返し、フレームサイズが固定だった。コマ 4 では `_reset_func_state()` を実装し、宣言された変数の数に応じて可変にする。

7 alloc_local の役割

`self.alloc_local(name)` は、新しいローカル変数にスタック上の位置を割り当てるメソッドである。

方針は次の通り。

処理	内容
<code>self._stack_offset</code> を 8 増やす	変数 1 個分の領域を確保したことにする
<code>self._locals[name]</code> にオフセットを登録する	変数名から位置を引けるようにする

たとえば、`a`, `b`, `c` を順に割り当てると、次のようになる。

変数	オフセット
<code>a</code>	-24
<code>b</code>	-32
<code>c</code>	-40

オフセットは `s0` からの相対位置である。

8 rvalue と lvalue

変数を扱うときは、「値がほしい」のか「書き込み先のアドレスがほしい」のかを区別する必要がある。

たとえば、次の文を考える。

```
a = b + 1;
```

右辺の `b` は、変数 `b` に入っている値を読む必要がある。一方、左辺の `a` は、値を読むのではなく、代入先として `a` のアドレスが必要である。

場所	必要なもの
右辺の <code>b</code>	<code>b</code> に保存されている値
左辺の <code>a</code>	<code>a</code> が置かれているメモリアドレス

この区別のために、次の 2 つのメソッドを使い分ける。

メソッド	役割
<code>self.codegen(node)</code>	式の値を計算し、結果を <code>a0</code> に置く
<code>self.codegen_lval(node)</code>	代入先のアドレスを計算し、結果を <code>a0</code> に置く

この回の `codegen_lval()` は、`'Var'` だけ対応すればよい。ポインタや配列に対する `lvalue` は後の回で扱う。

9 変数参照の生成

`'Var'` を値として読む場合は、次の 2 段階で処理する。

1. `self.codegen_lval(node)` で変数のアドレスを `a0` に入れる
2. `ld a0, 0(a0)` で、そのアドレスから値を読み込む

たとえば、`a` が `s0 - 24` にあるなら、次のような命令になる。

```
addi a0, s0, -24
ld a0, 0(a0)
```

1 行目で `a` のアドレスを作り、2 行目でそのアドレスに保存されている値を読む。

10 代入の生成

`a = 3;` のような代入では、左辺のアドレスと右辺の値の両方が必要になる。

生成パターンは次の通り。

1. 左辺のアドレスを `self.codegen_lval()` で計算する
-> `a0` に左辺のアドレスが入る
2. 左辺のアドレスをスタックに退避する

3. 右辺を `self.codegen()` で計算する
-> `a0` に右辺の値が入る
4. 退避していた左辺のアドレスを `a1` に戻す
5. `sd a0, 0(a1)` で、右辺の値を左辺のアドレスへ保存する

代入後、`a0` には右辺の値を残しておく。C では、代入式 `a = 3` 自体の値は `3` である。

11 gen_stmt の役割

`self.gen_stmt(node)` は、文ノードを受け取り、`match node.kind` で種類ごとに handler を呼ぶ。

ノード種別	呼ばれる handler	処理
'Decl'	<code>gen_stmt_Decl</code>	変数宣言。初期化子があれば代入として処理する
'ExprStmt'	<code>gen_stmt_ExprStmt</code>	<code>node.operand</code> を <code>self.codegen()</code> する
'Return'	<code>gen_stmt_Return</code>	<code>node.operand</code> を <code>self.codegen()</code> し、戻り値を <code>a0</code> に置く

'Block' の処理 (`body.stmts` を順に `gen_stmt()`) はコマ 3 の `_emit_func_body` がすでに行っている。

スケルトンでは dispatcher 部分があらかじめ書かれており、handler だけが TODO になって

いる。

12 編集するファイル

- mycc.py

コマ 3 の `Codegen03` を `importlib` で継承した `Codegen04` を実装する。

実装対象	役割
<code>alloc_local(name)</code>	変数名にスタック上の位置を割り当てる
<code>codegen_lval_Var(node)</code>	代入先のアドレス (<code>s0 + offset</code>) を <code>a0</code> に置く
<code>codegen_Var(node)</code>	左辺値アドレスを取得後 <code>ld</code> で値を読む
<code>codegen_Assign(node)</code>	左辺のアドレスを保存 → 右辺計算 → 保存
<code>_codegen_binary_value(node, op)</code>	コマ 3 の TODO を埋める（二項演算共通処理）
<code>codegen_Neg(node)</code>	コマ 3 の TODO を埋める
<code>codegen_Add</code> ~ <code>codegen_Mod</code>	コマ 3 の TODO を埋める
<code>gen_stmt_Decl(node)</code>	初期化子があれば代入文として処理
<code>gen_stmt_ExprStmt(node)</code>	式文を処理
<code>gen_stmt_Return(node)</code>	コマ 3 の TODO を埋める
<code>collect_decls_Decl(node)</code>	<code>alloc_local</code> を呼ぶ
<code>_reset_func_state</code>	関数ごとに変数表を初期化し、 <code>frame_size</code> を計算
<code>_emit_func_prologue</code> / <code>_emit_func_body</code> / <code>_emit_func_epilogue</code>	コマ 3 の TODO を埋める

コマ 3 で未実装だった TODO も合わせて埋める。

13 tests/

ファイル	内容	期待値
target.c	<code>a = 3; b = 5; c = a + b; return c;</code>	8
reassign.c	<code>x = 10; y = x * 2 + 3; x = y - x; return x;</code>	13
init.c	<code>int a; a = 42; return a;</code>	42
chain_assign.c	<code>a = 5; b = a; a = b + 1; return a;</code>	6

各ファイルでは、代入の前に `int a;` のようなローカル変数宣言が書かれている。

single.c | 1 変数への代入と参照 | 対応する .ans を参照 |

add_vars.c | 複数変数の加算 | 対応する .ans を参照 |

init.c | 初期化に近い単純代入 | 対応する .ans を参照 |

reassign.c | 再代入 | 対応する .ans を参照 |

expr_chain.c | 変数を含む式の連鎖 | 対応する .ans を参照 |

multi_expr.c | 複数変数と複数式 | 対応する .ans を参照 |

14 テスト

```
python3 scaffold/test_runner.py sessions/04_variables
```

tests/target.c がコンパイルでき、終了コード 8 になれば基本形は成功。

個別に動かす場合は、次のようにする。

```
python3 sessions/04_variables/mycc.py sessions/04_variables/tests/target.c \
| riscv64-linux-gnu-gcc -x assembler -static - -o out

qemu-riscv64 ./out
echo $?
```

15 注意

この回では、同じ関数内で同じ名前の変数を 2 回宣言するケースは扱わない。

コマ 4 ではローカル変数の数に応じて可変フレームを使う。`_reset_func_state()` で `collect_decls()` を呼び、`align_to(self._stack_offset, 16)` で 16 バイト境界に調整する。